SmartGreenAI: Radish Crop

Team / organization	RadishSoftware — 4 members
Course project	Agile Project Management — 7th semester
Source document	Project 1.pdf — IoT-Based Precision Agriculture and AI Decision-Support System for Greenhouse Radish Production
Capacity	4 members × 6.25 h/week × 14 weeks = 350 h (Sprint 0 + Sprints 1–4)

1. Project Description and Scope

SmartGreenAI: Radish Crop is a precision-agriculture prototype that monitors and optimizes the environmental and irrigation conditions of a greenhouse used to grow radishes. It joins five layers into one platform: a network of IoT sensors, controllable actuators, a cloud backend, a web dashboard, and an Artificial Intelligence component that turns the collected data into recommendations.

The system continuously measures temperature, humidity, light, CO_2, water-tank level, soil moisture and pH; sends those readings to the cloud; compares them against configurable thresholds; and generates alerts and irrigation recommendations. The producer can then irrigate manually or let the system do it automatically. The guiding principle stated in the project brief is decision support, not autonomy: the platform informs and recommends, and the radish producer decides.

1.1 In scope

- Five-layer architecture: IoT sensing, actuation, cloud/API |, web application and AI decision support.
- Three user roles with different functionality: Radish Producer, Administrator and Super Administrator.
- Real-time and historical monitoring, configurable threshold alerts, manual and automatic actuator control.
- An AI component that analyzes historical data, detects anomalies and recommends irrigation with a plain-language explanation.
- A demonstrable end-to-end cycle: sensor reading → cloud storage → alert / AI analysis → recommendation → actuator response.

1.2 Out of scope for this iteration

- Multi-tenant operation for several farms or clients (kept in the backlog as Won't items).
- Fully autonomous control with no human in the loop.
- Integrations with paid third-party weather or notification providers.

1.3 Delivery plan

After a Sprint 0 initiation phase (architecture, technology selection, repository and the first IoT proof of concept), the work is delivered in four sprints: Sprint 1 lasts 2 weeks (50 h) and Sprints 2, 3 and 4 last 3 weeks each (75 h each), for a total of 275 h of development capacity.

2. Product Backlog

2.1 What each MoSCoW priority means here

- Must — the prototype is not the promised product without it.
- Should — real value, planned for a later iteration.
- Could — nice to have, built only if everything else lands and there is time.
- Won't — explicitly out of scope this iteration, kept in the backlog so the decision stays visible.

2.2 Composition of the backlog

Item type	Count	Priority split
User stories	43	21 Must · 16 Should · 6 Could
Research spikes	3	3 Must (two already done in Sprint 0)
Technical improvements	5	1 Must (done in Sprint 0) · 2 Should · 2 Could
Defects	0	None yet — nothing has been built
Total	51	26 items are committed to Sprints 1–4

2.3 The backlog

Estimates use story points (Fibonacci) once an item has been refined, and a T-shirt size (S / M / L / XL) while it is still coarse. Only the 26 items committed to a sprint are fully refined — that is deliberate, not an omission.

Every backlog item carries acceptance criteria written in Given / When / Then form. They are the contract the Product Owner uses in the Sprint Review to accept or reject an item, and the input the team uses to write the test cases for system validation and testing. Criteria are derived directly from the functional and non-functional requirements of the source brief; the Traceability column records which requirement, use case or section each item answers to.
Items rated Won't carry a deferral note and an entry condition instead of criteria, because writing criteria for work the team has explicitly decided not to do this iteration would be waste.

ID	Item	MoSCoW	Estimate	Sprint	Acceptance criteria	Status
Epic A — Authentication & Access Control						
AUTH-01	As a Radish Producer, Administrator, or Super Administrator, I want to log in with my email/username and password, so that I can securely access only the functionality assigned to my role.	Must	5 sp	Sprint 2	Given a registered and active user, when they submit a valid email/username and password, then a session token is issued and they land on the dashboard for their role. Given invalid credentials, when login is attempted, then a generic error is shown that does not reveal which field failed, and no session is created. Given an idle session past the configured timeout, when the user makes a request, then it is rejected and re-authentication is required.	
AUTH-02	As a Radish Producer, Administrator, or Super Administrator, I want the system to reject access to functionality my assigned role does not include, so that agricultural, administrative and platform data stay isolated from unauthorized users.	Must	3 sp	Sprint 2	Given a Radish Producer session, when a request reaches an Administrator or Super Administrator route, then the server returns 403 and the action is not performed. Given any screen, when it renders, then options outside the user's role are absent from the menu — hiding in the UI alone is not accepted as the control. Given a request with a missing or expired token, when it reaches a protected endpoint, then it is rejected with 401.	

AUTH-03	As an Administrator, I want to activate or deactivate agricultural user accounts, so that access can be revoked the moment someone leaves or is suspended.	Should		—	Given an active agricultural user, when an Administrator deactivates the account, then the user can no longer log in and any open session is invalidated. Given a deactivated user, when the Administrator reactivates them, then login works again and their history remains intact. Given a deactivation or reactivation, when it completes, then the event is recorded with actor and timestamp.	
AUTH-04	As a Super Administrator, I want to define roles and permissions, so that each type of user only ever sees the functionality relevant to their job.	Should	M	—	Given a Super Administrator, when they create or edit a role, then they can select its permissions from the available list and save the definition. Given a user whose role changed, when they next sign in, then only the functionality granted by the new permission set is reachable. Given a role still assigned to at least one user, when deletion is attempted, then it is blocked with an explanatory message.	
AUTH-05	As a Super Administrator, I want to centrally store user activity, alerts and actuator events, so that a complete audit trail exists for every action taken on the platform.	Must	5 sp	Sprint 4	Given logins, alerts, actuator events and AI recommendations, when they occur, then each is written to a central log with actor, action and timestamp. Given a query by date and event type, when it runs, then the matching entries are returned. Given the application, when it is used, then existing log entries cannot be modified or deleted.	
Epic B — Greenhouse & Crop Configuration						
CONF-04	As an Administrator, I want to register IoT devices and the sensors attached to them, so that every incoming measurement can be identified and routed to the right greenhouse/area.	Must	8 sp	Sprint 1	Given an Administrator, when they register an IoT device with a unique identifier and assign it to a greenhouse, then the device appears in the configuration list. Given a registered device, when a sensor is attached with type, unit and area, then the sensor is uniquely identifiable and every incoming measurement is routed to that area. Given a device or sensor identifier that already exists, when registration is attempted, then it is rejected with a duplicate error.	
CONF-05	As an Administrator, I want to register the actuators available in a greenhouse (pump, fans, shade, lighting), so that the system only ever tries to control equipment that is actually installed.	Must	3 sp	Sprint 2	Given an Administrator, when they register an actuator with type (pump / fan / shade / light), greenhouse-area and control channel, then it becomes available in the control screens. Given an actuator that is not registered, when a control screen opens, then it does not appear and cannot be commanded. Given an actuator with existing history, when it is deactivated, then it disappears from the control screen but its event history is preserved.	
CONF-06	As a developer, I want to configure sensor thresholds for the greenhouse, so that the alert engine knows what counts as abnormal for this crop.	Must	3 sp	Sprint 2	Given a sensor type and an area, when the Administrator sets minimum and maximum thresholds, then the values are stored and read by the alert engine. Given a minimum greater than or equal to the maximum, when saving, then it is rejected with a validation message. Given a threshold change, when the next measurement arrives, then it is evaluated against the new value with no redeployment.	
CONF-07	As an Administrator, I want to configure alert rules, including severity, so that the right people are notified with the right urgency instead of one flat alert type.	Should	M	—	Given an Administrator, when they create an alert rule, then they can specify the sensor, the condition and a severity of Low, Medium or High. Given a measurement that satisfies a rule, when the alert is raised, then it carries the severity configured in that rule. Given several rules firing at once, when the alert is built, then the highest severity prevails and no duplicate is raised for the same condition.	
CONF-08	As an Administrator, I want to configure irrigation parameters (target moisture, run time), so that automatic irrigation matches this crop's actual water needs.	Must	3 sp	Sprint 2	Given an area, when the Administrator sets a target soil moisture and a maximum run time, then automatic irrigation uses those values.	

					Given a negative value or a zero run time, when saving, then it is rejected with a validation message. Given a change to the parameters, when the next automatic cycle runs, then it uses the updated values.	
CONF-09	As an Administrator, I want to configure how notifications are delivered (e.g. in-app, email, SMS), so that alerts reach the radish producer through a channel they actually check.	Could	M	—	Given the available channels (in-app, email, SMS), when the Administrator enables one or more, then alerts are delivered through those channels. Given a channel that is unavailable or not configured, when it is selected, then it is rejected with an explanatory message. Given a delivery failure, when it happens, then it is logged and the in-app alert is still shown.	
Epic C — IoT Monitoring & Data Collection						
MON-01	As a Radish Producer, I want to collect temperature and humidity measurements from the greenhouse, so that environmental conditions can be monitored continuously.	Must	3 sp	Sprint 1	Given a gateway with temperature and humidity sensors, when the configured interval elapses, then both readings are taken and placed in the payload. Given a reading, when it is produced, then it falls inside the sensor's documented range and carries its unit. Given a failed read, when it occurs, then it is logged, an explicit error value is reported and the firmware loop continues.	
MON-02	As a Radish Producer, I want soil-moisture measurements to be collected automatically, so that I can determine whether irrigation is required without walking the greenhouse.	Must	2 sp	Sprint 1	Given an area with a soil-moisture probe, when the reading cycle runs, then the moisture value is collected in the same payload as temperature and humidity. Given a disconnected probe, when the read is attempted, then an explicit error value is reported instead of 0. Given a valid reading, when it is transmitted, then it carries its sensor identifier and the area it belongs to.	
MON-03	As an Administrator, I want IoT measurements to be transmitted to the cloud, so that information can be stored and accessed remotely instead of staying on the device.	Must	5 sp	Sprint 1	Given a gateway with connectivity, when a payload is ready, then it is published to the cloud endpoint carrying device ID, sensor ID, value, unit and timestamp. Given a transmission failure, when it occurs, then the payload is buffered locally and retried; nothing is discarded silently. Given restored connectivity, when the buffer is flushed, then the readings keep their original timestamps.	
MON-04	As a Radish Producer, I want measurements to be stored with a timestamp and sensor ID, so that I can analyze greenhouse conditions over time and trust which sensor produced which value.	Must	3 sp	Sprint 1	Given a measurement received in the cloud, when it is persisted, then it stores sensor ID, greenhouse-area, value, unit and UTC timestamp. Given a query by sensor and date range, when it runs, then only measurements inside that range are returned, ordered in time. Given a duplicated delivery of the same reading, when it is processed, then no record is duplicated, overwritten or lost.	
MON-06	As a Radish Producer, I want to see current sensor measurements on demand, not only on a fixed refresh, so that I know the real-time state of my greenhouse whenever I check.	Should	S	—	Given the dashboard, when the user requests a refresh, then the latest stored values are fetched without reloading the whole page. Given a completed refresh, when the values render, then the time elapsed since each reading is displayed. Given an unreachable backend, when a refresh is requested, then an explicit error is shown and the previous values stay visible, marked as outdated.	
MON-07	As a Radish Producer, I want to also see water-tank level when the sensor is installed, so that I can fill the water tank when needed	Could	M	—	Given a greenhouse with a sensor that can track the water tank level, a notification can be sent to the Radish producer to refill the water tank.	

MON-08	As a Radish Producer, I want to view a history of past alerts, so that I can review recurring problems and judge whether a fix actually worked.	Should	S	—	Given generated alerts, when the history is queried, then they are listed with sensor, value, threshold, severity, timestamp and resolution state. Given a filter by date range, severity or sensor, when it is applied, then only matching alerts are returned. Given a resolved alert, when it is displayed, then its resolution date is shown and the record is never deleted.	
Epic D — Dashboard & Visualization						
DASH-01	As a Radish Producer, I want to see current sensor measurements on a dashboard, so that I can understand the current greenhouse conditions at a glance.	Must	7 sp	Sprint 2	Given a producer with an assigned greenhouse, when the dashboard opens, then the latest value per sensor is shown with unit, area and time since the reading. Given a reading older than the configured interval, when it is displayed, then it is flagged as stale. Given the prototype dataset, when the page loads, then it renders in approximately 3 seconds or less.	
DASH-02	As a Radish Producer, I want to visualize historical sensor data as graphs, so that I can identify environmental trends instead of only isolated readings.	Must	8 sp	Sprint 3	Given a selected sensor and date range, when the chart is requested, then a line chart of the stored measurements is drawn with labelled units and time axis. Given a range containing no data, when it is requested, then an explicit message is shown instead of an empty chart. Given a range with a large number of points, when it renders, then the series is aggregated or sampled so the chart stays legible.	
DASH-03	As a Radish Producer, I want sensor readings, alerts, actuator status and AI recommendations combined into one dashboard, so that I can manage the whole greenhouse from a single screen instead of switching between tools.	Must	5 sp	Sprint 4	Given a producer, when they open the integrated dashboard, then current measurements, active alerts, actuator status and the latest AI recommendation are visible on one screen. Given each section, when its data updates, then it refreshes without a full page reload. Given a phone screen, when the dashboard renders, then the layout stays usable with no horizontal scrolling.	
DASH-04	As an Administrator, I want a view of system statistics (device count, alert volume, uptime), so that I can assess overall platform health without reading raw logs.	Should	M	—	Given an Administrator, when they open the statistics view, then registered devices, connected devices, alert volume for the period and platform uptime are shown. Given a change of period, when it is applied, then every indicator is recalculated for that period. Given a period with no data, when it renders, then zeros are shown explicitly rather than blanks.	
DASH-05	As an Administrator, I want to view system logs, so that I can troubleshoot technical issues when something doesn't behave as expected.	Should	S	—	Given an Administrator, when they open the log view, then entries are listed with timestamp, level, component and message, newest first. Given filters by level and date range, when they are applied, then only matching entries are returned. Given a user without the Administrator role, when they request the view, then access is denied.	
Epic E — Alerts						
ALERT-01	As a Radish Producer, I want to receive an alert when environmental conditions exceed acceptable limits, so that I can take corrective action before the crop is affected.	Must	8 sp	Sprint 2	Given a configured threshold, when a measurement crosses it, then an alert is created within one polling cycle recording sensor, value, threshold and timestamp. Given a created alert, when the dashboard is open, then it becomes visible without a manual page refresh. Given a condition that persists, when further measurements keep breaching it, then no duplicate alert is raised until the condition clears.	

ALERT-02	As a system, I want to classify each alert by severity (Low / Medium / High), so that the radish producer can tell an urgent problem from a minor deviation.	Should	S	—	Given an alert being generated, when its severity is computed, then it is classified as Low, Medium or High according to the configured deviation. Given the alert list, when it renders, then severity is visually distinguishable and can be sorted on. Given a High-severity alert, when it is raised, then it is presented ahead of lower-severity ones.	
Epic F — Actuators & Irrigation Control						
ACT-02	As a Radish Producer, I want the system to activate irrigation automatically according to predefined conditions, so that irrigation can be managed efficiently even when I'm not watching the dashboard.	Must	5 sp	Sprint 2	Given automatic mode enabled and soil moisture below the configured target, when the rule is evaluated, then irrigation starts on its own. Given irrigation running automatically, when the target is reached or the maximum run time is hit, then it stops. Given the system in manual mode, when the condition is met, then automation is skipped and only manual commands act. Given an automatic action, when it is recorded, then it is distinguishable from a manual one.	
ACT-03	As a Radish Producer, I want to switch between automatic and manual operating mode, so that I retain final say over whether the system is allowed to act on its own.	Should	S	—	Given a producer, when they switch between automatic and manual mode, then the change takes effect immediately and is visible on the dashboard. Given a switch to manual while irrigation is running automatically, when it is confirmed, then the cycle stops or finishes under the producer's explicit control. Given a mode change, when it happens, then it is logged with user and timestamp.	
ACT-04	As a Radish Producer, I want to see the current status of each actuator (on/off), so that I know what equipment is running right now before I decide to intervene.	Must	2 sp	Sprint 2	Given registered actuators, when the dashboard renders, then each shows its state as on, off or unavailable. Given a manual or automatic action, when it completes, then the displayed state refreshes without a full page reload. Given an actuator that does not confirm its state, when it renders, then it shows unknown — never off.	
ACT-06	As a system, I want to record every actuator activation event (which actuator, when, triggered by whom/what), so that irrigation and ventilation actions are fully auditable.	Should	S	—	Given any actuator activation or deactivation, when it occurs, then a record is written with actuator, action, trigger (user or rule) and timestamp. Given the event log, when it is queried, then entries can be filtered by actuator and by date. Given the application, when it is used, then existing event records cannot be edited or deleted.	
Epic G — Artificial Intelligence & Decision Support						
AI-01	As a Radish Producer, I want the system to analyze historical sensor information, so that I can understand environmental patterns instead of only single readings.	Must	10 sp	Sprint 3	Given stored measurements for a selected period, when the analysis runs, then a trend indicator per variable is returned — rising, stable or falling — together with the period used. Given identical input, when the analysis is re-run, then the result is reproducible. Given gaps in the data, when they are detected, then they are reported in the result instead of being silently ignored.	
AI-02	As a Radish Producer, I want the system to identify unusual environmental conditions, so that potential problems can be detected early, before they become alerts.	Should	5 sp	Sprint 3	Given a sensor's recent pattern, when a measurement deviates from it, then it is flagged as an anomaly recording sensor, value, expected range and timestamp. Given a flagged anomaly, when the dashboard renders, then the producer sees it distinguished from a threshold alert. Given a value inside the expected pattern, when it is evaluated, then no anomaly is generated.	
AI-04	As a Radish Producer, I want AI to consider temperature, soil moisture and irrigation history together, so that recommendations are based on multiple factors, not one threshold in isolation.	Should	5 sp	Sprint 3	Given the recommendation logic, when it runs, then it consumes temperature, soil moisture and time since the last irrigation. Given a change in any one of those variables, when the logic re-runs, then the outcome is able to change.	

					Given a stored recommendation, when it is consulted, then the exact inputs used are visible.	
AI-05	As a Radish Producer, I want every AI recommendation to include a plain-language explanation, so that I understand why it was generated and can decide whether to trust it.	Should	3 sp	Sprint 3	Given any recommendation, when it is displayed, then it carries a plain-language explanation naming the measurements that drove it. Given the explanation, when a non-technical user reads it, then it contains no internal identifiers, formulas or codes. Given a recommendation with no explanation, when it is generated, then it is not displayed.	
AI-06	As a Radish Producer, I want each recommendation to show a confidence or certainty indicator, so that I can judge how much weight to give it before acting.	Could	S	—	Given a recommendation, when it is displayed, then it shows a confidence indicator (high / medium / low, or a percentage). Given sparse or incomplete data, when the recommendation is generated, then confidence drops and the reason is stated. Given the indicator, when it is shown, then the criterion behind it is documented and applied consistently across recommendations.	
AI-07	As a Radish Producer, I want to accept or reject an AI recommendation, so that the final irrigation decision stays under my control.	Should	3 sp	Sprint 4	Given a displayed recommendation, when the producer accepts or rejects it, then the decision is stored with user and timestamp. Given an accepted irrigation recommendation, when it is confirmed, then the irrigation action can be triggered from the same screen. Given a rejected recommendation, when it is filed, then it stays visible in the history with its outcome.	
AI-08	As a Radish Producer, I want to add my own free-text observations about the crop, so that my field knowledge (pests, visible stress, growth stage) is captured alongside the sensor data.	Could	M	—	Given a producer, when they record a free-text observation, then it is stored linked to the greenhouse-area with author and timestamp. Given existing observations, when the history is consulted, then they appear on the same timeline as the measurements. Given an observation, when it is saved, then only its author or an Administrator can edit or delete it.	
AI-09	As an Administrator, I want to configure the parameters used by the AI component, so that recommendations can be tuned to this specific greenhouse and crop cycle.	Could	M	—	Given an Administrator, when they modify the AI parameters (analysis window, sensitivity, moisture target), then subsequent recommendations use the new values. Given an out-of-range value, when saving is attempted, then it is rejected with a validation message. Given a parameter change, when it takes effect, then it is logged with actor, previous value and new value.	
Epic H — Data, APIs & System Integration						
SYS-01	As a system, I want to expose APIs for IoT devices and the web application, so that new devices or front-ends can integrate with the platform without custom wiring.	Must	2 sp	Sprint 1	Given the endpoints used by the IoT device and the web application, when the specification is published, then an OpenAPI/Swagger document describes every route, its parameters and its responses. Given the specification, when it is stored, then it is versioned in the repository alongside the code. Given the ingestion and query endpoints, when they are called, then they behave and return the status codes documented in the specification.	
SYS-04	As a Radish Producer, I want sensor measurements to automatically reach the cloud and generate alerts/recommendations end-to-end, so that the complete system keeps operating continuously without manual intervention.	Must	8 sp	Sprint 4	Given a new measurement and no human interaction, when it is emitted, then it reaches the cloud, is stored, is checked against thresholds, is analysed by the AI and produces an alert or a recommendation. Given the complete chain, when it is demonstrated, then every step is verifiable in the dashboard and in the logs. Given a broken link in the chain, when it fails, then a visible error is raised instead of failing silently.	
Epic I — Non-Functional Qualities, Testing & Delivery						

NFR-01	As a Radish Producer, I want the dashboard to respond within about 3 seconds under normal load, so that I can check my greenhouse quickly, including from a phone in the field.	Should	M	—	Given the prototype dataset, when the main dashboard operations are executed, then each responds in approximately 3 seconds or less. Given a measured response time above the target, when it is observed, then it is recorded as a defect with its evidence. Given the measurement, when it is taken, then the conditions (data volume, network, device) are documented so it can be reproduced.	
NFR-02	As a Super Administrator, I want the architecture to support adding new sensors, areas and greenhouses without a redesign, so that the platform can grow with the business instead of hitting a rebuild wall.	Could	L	—	Given a running deployment, when a new sensor, area or greenhouse is added, then it works with no schema change and no code redeployment. Given the architecture, when it is reviewed, then adding a sensor type does not require modifying the ingestion layer. Given the added capacity, when it is loaded, then existing queries keep their documented performance.	
NFR-03	As a user, I want my password to never be stored in plain text, so that my account can't be trivially compromised if the database is ever leaked.	Must	2 sp	Sprint 4	Given a password, when it is stored, then only a salted hash (bcrypt or argon2) is persisted. Given the database, the logs and the API responses, when they are inspected, then no plain-text password appears anywhere. Given pre-existing test accounts, when the change is deployed, then they are migrated or recreated under the new scheme.	
NFR-04	As an Administrator, I want IoT-to-cloud communication to be authenticated and encrypted, so that greenhouse data and controls can't be intercepted or spoofed.	Should	M	—	Given an IoT device, when it connects to the cloud, then it authenticates with a credential or token unique to that device. Given a payload in transit, when it is transmitted, then it travels over an encrypted channel (TLS) wherever the selected infrastructure supports it. Given an unauthenticated device, when it attempts to send data, then it is rejected and the attempt is logged.	

3. Sprint Backlog and Effort Estimation

The sprint backlog is the part of the product backlog the team commits to for one sprint, broken into tasks small enough to start the next day. Each task was estimated with Planning Poker in story points and then converted to hours for scheduling.

3.1 Capacity and commitment

Each sprint commits about 86 % of its capacity. The remaining 14 % is reserved on purpose for defects found during the sprint, refinement of upcoming items, and the ceremonies themselves. Once a sprint starts its backlog is frozen: new work goes to the product backlog for the next Sprint Planning, and only a critical defect may enter mid-sprint using the reserved hours.

Sprint	Dates	Weeks	Capacity	Committed	%	Items	Tasks	Reserved
Sprint 1	7 Sep – 18 Sep 2026	2	50 h	47 h (25 sp)	94 %	7	9	3 h
Sprint 2	21 Sep – 9 Oct 2026	3	75 h	67 h (39 sp)	89 %	9	12	8 h
Sprint 3	12 Oct – 30 Oct 2026	3	75 h	64 h (34 sp)	85 %	6	8	11 h
Sprint 4	2 Nov – 20 Nov 2026	3	75 h	61 h (26 sp)	81 %	5	8	14 h
Total	**7 Sep – 20 Nov 2026**	**11**	**275 h**	**239 h (124 sp)**	**86 %**	**27**	**37**	36 h

Sprint dates assume a start on Monday 7 September 2026 and must be confirmed with the real course calendar.

3.2 How the estimates were produced (Planning Poker)

1. The task is read aloud and the team confirms everyone shares the same idea of what "done" means for it.
2. Each member privately picks a Fibonacci card (1, 2, 3, 5, 8, 13) for relative effort, complexity and risk — not calendar time.
3. Everyone reveals at the same time. If the cards agree, or are within one step, that value is recorded.
4. If the spread is wide, the highest and the lowest estimator explain their reasoning and the team votes again until it converges.
5. Story points are converted to hours with a provisional mapping (1 sp ≈ 2–3 h, 2 sp ≈ 4–5 h, 3 sp ≈ 5–7 h, 5 sp ≈ 8–11 h, 8 sp ≈ 11–16 h), to be replaced by the team's real velocity after Sprint 1.

Note: the values below are a well-argued draft prepared for this deliverable. They should be re-run with the whole team in a real Planning Poker session.

3.3 Sprint 1 — Foundation: IoT Pipeline & Device Registration

Sprint goal: The sensors are up, running, and sending information to the system.

ID	Backlog item	Task	Points	Hours
PVZ-01	Cultivate the radish	Buy and plant the seeds.	1	2
		Build the greenhouse.	1	3
CONF-04	Register IoT devices & sensors (Must)	Design device/sensor schema blueprint (unique IDs, greenhouse/area FK)	3	5
		Build device & sensor registration API + admin UI	5	9
MON-01	Collect temperature & humidity (Must)	Implement sensor read + payload serialization on the IoT gateway	3	6
MON-02	Collect soil moisture (Must)	Add soil-moisture sensor read to the gateway payload	2	4
MON-03	Transmit measurements to the cloud (Must)	Implement MQTT/HTTP client on the Raspberry Pi como gateway	5	8
MON-04	Store measurements with timestamp (Must)	Build ingestion e ndpoint + write to time-series table	3	6
SYS-01	Expose APIs for IoT devices & web app (Must)	Define and publish the REST API contract (OpenAPI spec)	2	4
Total — 6 items, 7 tasks			23	47

3.4 Sprint 2 — Access Control, Dashboard, Alerts & Actuators

Sprint goal: To automate greenhouse monitoring and irrigation through real-time data tracking and smart alerts.

ID	Backlog item	Task	Points	Hours
AUTH-01	User login (Must)	Implement login endpoint + session/JWT handling	3	5
		Build login UI	2	3
AUTH-02	Role-based access enforcement (Must)	Implement server-side role guards on every route	3	6
CONF-05	Register actuators (Must)	Design actuator schema + registration API	3	5

CONF-06	Configure sensor thresholds (Must)	Build threshold configuration API + admin UI	3	6
CONF-08	Configure irrigation parameters (Must)	Build irrigation-parameter configuration UI/API	3	5
DASH-01	Current-measurements dashboard (Must)	Build "latest readings" API	2	4
		Build dashboard UI widgets	5	8
ALERT-01	Threshold alerts (Must)	Build alert engine (compare readings vs. thresholds)	5	8
		Build alert notification UI	3	5
ACT-02	Automatic irrigation (Must)	Build rule engine for automatic trigger	5	8
ACT-04	Actuator status view (Must)	Build actuator-status endpoint + UI badge	2	4
Total — 9 items, 12 tasks			39	67

3.5 Sprint 3 — History, Analysis & AI Recommendations

Sprint goal: The producer can see how conditions evolved and receives an AI-driven insight — anomaly flags, multi-variable analysis and a plain-language explanation — built from more than one variable.

ID	Backlog item	Task	Points	Hours
NF-01	Integrate the CO_2 sensor	Integrate the sensor the teacher gave us to the project with the thresholds it must follow	3	5
DASH-02	Historical graphs (Must)	Build historical-readings query API	3	6
		Build chart UI component	5	8
AI-01	Historical analysis (Must)	Build data-preprocessing pipeline	5	10
		Implement trend-analysis logic	5	10
AI-02	Anomaly detection (Should)	Implement anomaly-detection rules/model	5	10
AI-04	Multi-variable analysis (Should)	Extend the recommendation engine to weigh multiple variables	5	8
AI-05	Explainable recommendation (Should)	Generate the natural-language explanation text	3	7
Total — 6 items, 8 tasks			34	64

3.6 Sprint 4 — Integration, Validation & Deployment

Sprint goal: *Every part built so far runs as one secure, tested platform, and the team can perform the brief's acceptance scenario live from a single dashboard.*

ID	Backlog item	Task	Points	Hours
AUTH-05	Centralized audit trail (Must)	Design unified event schema (user activity, alerts, actuator events)	2	5
		Implement audit-trail write layer + query API	3	8
DASH-03	Integrated dashboard (Must)	Merge sensor, alert, actuator and AI views into one dashboard	5	11
		Build responsive layout + cross-section navigation	3	7
AI-07	Accept/reject recommendation (Should)	Build accept/reject UI + store the producer's decision	3	7
SYS-04	End-to-end automatic operation (Must)	Wire the full pipeline: sensor → cloud → alert/AI → actuator	5	11
		Write integration tests for the end-to-end flow	3	7

NFR-03	Passwords never stored in plain text (Must)	Implement password hashing (bcrypt/argon2) + migration check	2	5
Total — 5 items, 8 tasks			26	61

4. Scrum Roles and Ceremonies

The team has 4 members. Because the team is small, the Product Owner and the Scrum Master also take development tasks, which is normal at this size. Names are still to be confirmed with the team.

Role	Person	Dedication	Main responsibilities
Product Owner	Alexander	Part-time (also develops)	Owns and orders the Product Backlog. Approves acceptance criteria before an item enters a sprint. Represents the radish producer in planning. Accepts or rejects work in the Sprint Review.
Scrum Master	Xavier	Part-time (also develops)	Facilitates Planning, Daily Stand-up, Review and Retrospective. Keeps the meeting record. Removes blockers and protects the sprint from mid-sprint scope changes. Tracks capacity vs. commitment.
Developer — IoT & Backend	Fernando	6.25 h/week	Sensor and gateway firmware, cloud ingestion API, database schema. Owns Epic B (Configuration) and Epic C (IoT Monitoring).
Developer — Web & Dashboard	Emiliano	6.25 h/week	Web dashboard, authentication UI, alerts UI, actuator controls. Owns the UI work in Epics A, D, E and F.
Developer — AI & Data	Fernando	6.25 h/week	Historical analysis, anomaly detection, recommendation engine, explanation generation. Owns Epic G (Artificial Intelligence).

4.1 Who owns each artifact

- The Product Owner owns the product backlog: what is in it, in what order, and the acceptance criteria each item carries.
- The Development Team owns the sprint backlog: what it commits to, how items break into tasks, and the estimates behind them.
- The Scrum Master owns neither backlog — they own the process, the meeting record and the sprint freeze rule.

4.2 Ceremonies

- Sprint Planning — the Product Owner presents the ordered backlog, the team estimates with Planning Poker and commits; the output is the sprint backlog and the sprint goal.
- Backlog Refinement — a standing session mid-sprint to split, clarify, estimate and re-rank upcoming items so the next planning has ready work waiting.
- Daily Stand-up — 15 minutes with the Development Team: what was done, what is next, what is blocked.
- Sprint Review — the team demonstrates the increment to stakeholders and the Product Owner accepts or rejects each item.
- Sprint Retrospective — the team chooses one concrete process change to apply in the next sprint.

4.3 Definition of Done

An item counts as done, in any sprint, only when all of the following are true:

- The code is merged into the main branch and the build and test suite pass.
- The acceptance criteria are verified by someone other than the author.

- The Product Owner has accepted the item in the Sprint Review.
- It runs in the shared demo environment, not only on a developer's machine.
- Documentation and the API spec are updated where the change touched them.
- No known critical defect remains open against it.

5. Meeting Record

Every ceremony is recorded with the template below, saved as one file per meeting (for example, 2026-09-07-sprint-planning). The Scrum Master is responsible for filling it in and sharing it with the team on the same day.

Ceremony		Sprint	
Date and time		**Facilitator**	
Attendees		**Absent (reason)**	

Agenda	
Decisions taken	
Sprint goal / scope changes	
Next meeting	

Action items

Action item	Owner	Due date	Status

Impediments raised

Impediment	Blocking whom / which item	Resolution owner

Link a ClickUp: https://app.clickup.com/9017994214/v/l/s/90177463298